[69] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *J. Mach. Learn. Res.* 21 (2020), 140:1–140:67. http://jmlr.org/papers/v21/20-074.html
[70] Talia Ringer. 2021. *Proof Repair.* Ph. D. Dissertation. University of Washington.
[71] Talia Ringer, Karl Palmskog, Ilya Sergey, Milos Gligoric, Zachary Tatlock, et al. 2019. QED at large: A survey of engineering of formally verified software. *Foundations and Trends® in Programming Languages* 5, 2-3 (2019), 102–281.
[72] Talia Ringer, Alex Sanchez-Stern, Dan Grossman, and Sorin Lerner. 2020. REPLica: REPL Instrumentation for Coq Analysis. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs* (New Orleans, LA, USA) *(CPP 2020)*. Association for Computing Machinery, New York, NY, USA, 99–113. https://doi.org/10.1145/3372885.3373823
[73] Ripon K. Saha, Yingjun Lyu, Hiroaki Yoshida, and Mukul R. Prasad. 2017. ELIXIR: Effective object oriented program repair. In *ASE*. 648–659.
[74] Seemanta Saha, Ripon K. Saha, and Mukul R. Prasad. 2019. Harnessing Evolution for Multi-Hunk Program Repair. In *ACM/IEEE International Conference on Software Engineering (ICSE)* (29–31). Montreal, QC, Canada, 13–24. https://doi.org/10.1109/ICSE.2019.00020
[75] Alex Sanchez-Stern, Yousef Alhessi, Lawrence Saul, and Sorin Lerner. 2020. Generating Correctness Proofs with Neural Networks. In *Proceedings of the 4th ACM SIGPLAN International Workshop on Machine Learning and Programming Languages* (London, UK) *(MAPL 2020)*. Association for Computing Machinery, New York, NY, USA, 1–10. https://doi.org/10.1145/3394450.3397466
[76] Alex Sanchez-Stern, Emily First, Timothy Zhou, Zhanna Kaufman, Yuriy Brun, and Talia Ringer. 2023. Passport: Improving Automated Formal Verification Using Identifiers. *ACM TOPLAS* (2023).
[77] Noam Shazeer. 2019. Fast Transformer Decoding: One Write-Head is All You Need. *CoRR* abs/1911.02150 (2019). arXiv:1911.02150 http://arxiv.org/abs/1911.02150
[78] Edward K. Smith, Earl Barr, Claire Le Goues, and Yuriy Brun. 2015. Is the Cure Worse than the Disease? Overfitting in Automated Program Repair. In *ESEC/FSE*. 532–543. https://doi.org/10.1145/2786805.2786825
[79] Jianlin Su, Yu Lu, Shengfeng Pan, Bo Wen, and Yunfeng Liu. 2021. RoFormer: Enhanced Transformer with Rotary Position Embedding. *CoRR* abs/2104.09864 (2021). arXiv:2104.09864 https://arxiv.org/abs/2104.09864
[80] Shuyao Sun, Junxia Guo, Ruilian Zhao, and Zheng Li. 2018. Search-Based Efficient Automated Program Repair Using Mutation and Fault Localization. In *COMPSAC*, Vol. 1. 174–183. https://doi.org/10.1109/COMPSAC.2018.00030
[81] The Coq Development Team. 2017. Coq, v.8.7. https://coq.inria.fr.
[82] Haoye Tian, Kui Liu, Abdoul Kader Kaboré, Anil Koyuncu, Li Li, Jacques Klein, and Tegawendé F. Bissyandé. 2020. Evaluating Representation Learning of Code Changes for Predicting Patch Correctness in Program Repair. In *ASE*. https://doi.org/10.1145/3324884.3416532
[83] Yuchi Tian and Baishakhi Ray. 2017. Automatically diagnosing and repairing error handling bugs in C. In *European Software Engineering Conference and ACM SIGSOFT International Symposium on Foundations of Software Engineering (ESEC/FSE)*. Paderborn, Germany, 752–762.
[84] Aäron van den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves, Nal Kalchbrenner, Andrew W. Senior, and Koray Kavukcuoglu. 2016. WaveNet: A Generative Model for Raw Audio. *CoRR* abs/1609.03499 (2016). arXiv:1609.03499 http://arxiv.org/abs/1609.03499
[85] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *NeurIPS*.
[86] Ke Wang, Rishabh Singh, and Zhendong Su. 2018. Search, align, and repair: Data-driven feedback generation for introductory programming exercises. In *PLDI*. 481–495. https://doi.org/10.1145/3296979.3192384
[87] Shangwen Wang, Ming Wen, Bo Lin, Hongjun Wu, Yihao Qin, Deqing Zou, Xiaoguang Mao, and Hai Jin. 2020. Automated Patch Correctness Assessment: How Far Are We?. In *ASE*. Association for Computing Machinery, 968–980. https://doi.org/10.1145/3324884.3416590
[88] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Ed H. Chi, Quoc Le, and Denny Zhou. 2022. Chain of Thought Prompting Elicits Reasoning in Large Language Models. *CoRR* abs/2201.11903 (2022). arXiv:2201.11903 https://arxiv.org/abs/2201.11903
[89] Westley Weimer, Zachary P. Fry, and Stephanie Forrest. 2013. Leveraging Program Equivalence for Adaptive Program Repair: Models and First Results. In *IEEE/ACM International Conference on Automated Software Engineering (ASE)*. Palo Alto, CA, USA, 356–366.
[90] Ming Wen, Junjie Chen, Rongxin Wu, Dan Hao, and Shing-Chi Cheung. 2018. Context-Aware Patch Generation for Better Automated Program Repair. In *ACM/IEEE International Conference on Software Engineering (ICSE)*. Gothenburg, Sweden, 1–11. https://doi.org/10.1145/3180155.3180233
[91] Minchao Wu, Michael Norrish, Christian Walder, and Amir Dezfouli. 2021. TacticZero: Learning to Prove Theorems from Scratch with Deep Reinforcement Learning. In *Neural Information Processing Systems*. https://arxiv.org/abs/2102.09756
[92] Yuhuai Wu, Albert Q. Jiang, Wenda Li, Markus N. Rabe, Charles Staats, Mateja Jamnik, and Christian Szegedy. 2022. Autoformalization with Large Language Models. *CoRR* abs/2205.12615 (2022). https://doi.org/10.48550/ARXIV.2205.12615
[93] Qi Xin and Steven P. Reiss. 2017. Identifying Test-suite-overfitted Patches through Test Case Generation. In *ISSTA*. 226–236. https://doi.org/10.1145/3092703.3092718
[94] Deheng Yang, Yuhua Qi, and Xiaoguang Mao. 2018. Evaluating the Strategies of Statement Selection in Automated Program Repair. In *SATE*. Springer. https://doi.org/10.1007/978-3-030-04272-1_3
[95] Jinqiu Yang, Alexey Zhikhartsev, Yuefei Liu, and Lin Tan. 2017. Better test cases for better automated program repair. In *ESEC/FSE*. 831–841. https://doi.org/10.1145/3106237.3106274
[96] Kaiyu Yang and Jia Deng. 2019. Learning to prove theorems via interacting with proof assistants. In *International Conference on Machine Learning*. PMLR, 6984–6994.
[97] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. 2011. Finding and understanding bugs in C compilers. In *ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. San Jose, CA, USA, 283–294. https://doi.org/10.1145/1993498.1993532
[98] Michihiro Yasunaga and Percy Liang. 2021. Break-it-fix-it: Unsupervised learning for program repair. In *International Conference on Machine Learning (ICML)*. PMLR, 11941–11952.
[99] He Ye, Matias Martinez, and Martin Monperrus. 2021. Automated patch assessment for program repair at scale. *EMSE* 26, 2 (2021).
[100] Zhongxing Yu, Matias Martinez, Benjamin Danglot, Thomas Durieux, and Martin Monperrus. 2019. Alleviating patch overfitting with automatic test generation: A study of feasibility and effectiveness for the Nopol repair system. *EMSE* 24, 1 (2019), 33–67. https://doi.org/10.1007/s10664-018-9619-4
[101] Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. 2022. miniF2F: A cross-system benchmark for formal Olympiad-level mathematics. In *ICLR*.
[102] Qihao Zhu, Zeyu Sun, Yuan an Xiao, Wenjie Zhang, Kang Yuan, Yingfei Xiong, and Lu Zhang. 2021. A syntax-guided edit decoder for neural program repair. In *ESEC/FSE*. 341–353. https://doi.org/10.1145/3468264.3468544

A EXAMPLES OF PROOF GENERATION WITH CONTEXT

We provide a number of examples that the model using context could solve but the plain proof generation model could not. We determined the lists of problems each model could solve, computed their difference, and then sampled 5 examples uniformly at random. For examples that had multiple correct proofs generated by the model, we selected one at random. We modified whitespace in the examples to make them more readable with the reduced line length. Further, we truncated the examples on the left to help with readability, but we inspected also the full context to ensure that our conclusions below are not affected. Each example consists of the "context and problem statement", the "ground truth proof", and the "generated proof".

We can observe in examples 1, 3, and 5 that the model readily **copies and adapts** proofs that exist in its context. In example 2, the model made use of a premise that did not occur in its context, which happened to also be used by the ground truth proof, but with a different tactic. In example 4, the model found a simpler proof that did not occur like this in the context.

A.1 Example 1

Context and problem statement:

```
lemma (in Interpretation) InterpExprWellDefined:
  "L\<lbrakk>Vx : A \<turnstile> e :
    B\<rbrakk> \<rightarrow> i \<Longrightarrow>
      Sig iS \<triangleright> Vx :
        A  \<turnstile> e : B"
```

```
  apply (rule Interp.cases)
  by auto

lemma (in Interpretation) WellDefined:
  "L\<lbrakk>\<phi>\<rbrakk> \<rightarrow> i
    \<Longrightarrow> Sig iS \<triangleright> \<phi>"
  apply(rule Interp.cases)
  by (auto simp add: InterpExprWellDefined)

lemma (in Interpretation) Bool:
  "L\<lbrakk>\<phi>\<rbrakk> \<rightarrow>
    (IBool i) \<Longrightarrow> \<exists> A B e d .
      \<phi> = (Vx : A \<turnstile> e \<equiv> d : B)"
```

Ground truth proof:

```
apply(rule Interp.cases)
by auto
```

Generated proof:

```
apply (rule Interp.cases)
by (auto simp add: InterpExprWellDefined)
```

A.2 Example 2

Context and problem statement:

```
lemma Aeta_expand_0[simp]:
  "Aeta_expand 0 e = e"
  by transfer simp

lemma Aeta_expand_inc[simp]:
  "Aeta_expand (inc\<cdot>n) e =
    (Lam [fresh_var e].
      Aeta_expand n (App e (fresh_var e)))"
  apply (simp add: inc_def)
  by transfer simp

lemma subst_Aeta_expand:
  "(Aeta_expand n e)[x::=y] =
    Aeta_expand n e[x::=y]"
```

Ground truth proof:

```
by transfer (rule subst_eta_expand)
```

Generated proof:

```
by transfer (simp add: subst_eta_expand)
```

A.3 Example 3

Context and problem statement:

```
subsubsection \<open>Store and Assignment\<close>

type_synonym \'a store = "string  \<Rightarrow> \'a"

definition gets ::
  "string \<Rightarrow> (\'a store \<Rightarrow> \'a)
    \<Rightarrow> \'a store rel" ("_ ::= _" [70, 65] 61)
  where "v ::= e = {(s, s(v := e s)) |s. True}"

lemma H_assign:
  "rel_kat.H \<lceil>\<lambda>s.
    P (s (v := e s))\<rceil> (v ::= e) \<lceil>P\<rceil>"
  by (auto simp:
    gets_def
    rel_kat.H_def
    rel_kat.t_op_def
    rel_at_def)

lemma H_assign_var:
  "(\<forall>s. P s \<longrightarrow> Q (s (v := e s)))
    \<Longrightarrow>
    rel_kat.H
      \<lceil>P\<rceil> (v ::= e) \<lceil>Q\<rceil>"
```

Ground truth proof:

```
by (auto simp:
  gets_def
  rel_kat.H_def
  rel_kat.t_op_def
  rel_at_def)
```

Generated proof:

```
by (auto simp:
  gets_def
  rel_kat.H_def
  rel_kat.t_op_def
  rel_at_def)
```

A.4 Example 4

Context and problem statement:

```
section \<open>Primitive words\<close>

text\<open>If a word w is not a non-trivial power of
some other word, we say it is primitive.\<close>

definition primitive :: "\'a list \<Rightarrow> bool"
  where "primitive u = (\<forall> r k.
      r\<^sup>@k = u \<longrightarrow> k = 1)"

lemma primI[intro]:
  "(\<And> r k. r\<^sup>@k = u \<Longrightarrow> k = 1)
```

```
  \<Longrightarrow> primitive u"
  by (simp add: primitive_def)

lemma prim_nemp:
  "primitive u \<Longrightarrow> u \<noteq> \<epsilon>"
```

Ground truth proof:

```
proof-
have "u = \<epsilon> \<Longrightarrow>
  \<epsilon>\<^sup>@0 = u"
by simp
thus "primitive u \<Longrightarrow>
  u \<noteq> \<epsilon>"
using primitive_def zero_neq_one
by blast
qed
```

Generated proof:

```
using primitive_def
by force
```

A.5 Example 5

Context and problem statement:

```
lemma drop_take_drop:
  "drop n (take m xs) @ drop m xs = drop (min n m) xs"
proof cases
  assume A: "\<not>m < n"
  then show ?thesis
    using drop_append[of n "take m xs" "drop m xs"]
  by (cases "length xs < n")
       (auto simp: not_less min_def)
qed (auto simp: min_def)

lemma not_distinct_decomp_min_prefix:
  assumes "\<not> distinct ws"
  shows "\<exists> xs ys zs y.
    ws = xs @ y # ys @ y # zs \<and>
    distinct xs \<and>
    y \<notin> set xs \<and>
    y \<notin> set ys "
proof -
  obtain xs y ys where
    "y \<in> set xs" "distinct xs" "ws = xs @ y # ys"
    using assms by (auto simp: not_distinct_conv_prefix)
  moreover then obtain xs\' ys\'
    where "xs = xs\' @ y # ys\'"
    by (auto simp: in_set_conv_decomp)
  ultimately show ?thesis by auto
qed

lemma not_distinct_decomp_min_not_distinct:
  assumes "\<not> distinct ws"
  shows "\<exists>xs y ys zs.
    ws = xs @ y # ys @ y # zs \<and>
    distinct (ys @ [y])"
```

Ground truth proof:

```
using assms
proof (induct ws)
  case (Cons w ws)
  show ?case
  proof (cases "distinct ws")
    case True
    then obtain xs ys where
        "ws = xs @ w # ys" "w \<notin> set xs"
      using Cons.prems
        by (fastforce dest: split_list_first)
    then have "distinct
        (xs @ [w])" "w # ws = [] @ w # xs @ w # ys"
      using \<open>distinct ws\<close> by auto
    then show ?thesis by blast
  next
    case False
    then obtain xs y ys zs
        where "ws = xs @ y # ys @ y # zs \<and>
               distinct (ys @ [y])"
      using Cons by auto
    then have "w # ws = (w # xs) @ y # ys @ y # zs \<and>
               distinct (ys @ [y])"
      by simp
    then show ?thesis by blast
  qed
qed simp
```

Generated proof:

```
proof -
  obtain xs y ys where
    "y \<in> set xs" "distinct xs" "ws = xs @ y # ys"
  using assms by (auto simp: not_distinct_conv_prefix)
  moreover then obtain xs' ys'
    where "xs = xs' @ y # ys'"
    by (auto simp: in_set_conv_decomp)
  ultimately
  show ?thesis by auto
qed
```